SmartInterview: 5-Member Presentation & Defense Plan

SmartInterview: 5-Member Team Presentation Script & Technical Defense Guide

Project Name: SmartInterview — Agentic Adaptive Technical Mock Interview Platform

Target Audience: Academic Evaluator / Madam / Project Guide

Format: 5 Team Members (20% Equal Allocation, ~3 to 4 Minutes per Member, Total: 18-20 Minutes)

System Status: Live & Verified (Backend: **`http://127.0.0.1:8000`**, Frontend: **`http://localhost:5174`**, MySQL: **`Port 3306`**, ChromaDB: **`402 Chunks`**)

Quick System Health & Architecture Summary

| Component | Status | Port / URL | Key Technology |

|---|---|---|---|

| **Backend REST API** | ■ LIVE (PID: 26300) | **`http://127.0.0.1:8000`** | FastAPI 0.115, Uvicorn, Pydantic |

| **API Docs (Swagger)** | ■ LIVE | **`http://127.0.0.1:8000/docs`** | Auto-generated OpenAPI v3 |

| **Frontend UI** | ■ LIVE (PID: 22032) | **`http://localhost:5174`** | React 18, Vite 6.1, Tailwind CSS |

| **Relational DB** | ■ LIVE (PID: 7084) | Port 3306 | MySQL 8.0, SQLAlchemy 2.0 ORM |

| **Vector DB** | ■ INDEXED | Local Storage (**`chroma_db/`**) | ChromaDB 0.6.3 (402 CS chunks) |

| **Local Embeddings** | ■ OFFLINE | Local CPU | SentenceTransformers **`all-MiniLM-L6-v2`** |

| **Generative LLM** | ■ CONNECTED | Cloud API | Groq Cloud **`openai/gpt-oss-120b`** |

| **Voice Stack** | ■ CACHED | In-Memory Stream | **`edge-tts`** (Christopher Neural) + Groq Whisper |

Master Team Allocation Matrix

`

[Member 1: Overview, Architecture & Auth] (0:00 - 3:30)

■

▼

[Member 2: Context Ingestion & Skill Gap] (3:30 - 7:00)

■

▼

[Member 3: RAG Grounding & Question Gen] (7:00 - 10:30)

■

▼

[Member 4: Bloom Adaptive State Machine] (10:30 - 14:00)

■

▼

[Member 5: Multi-Factor Scoring & Voice] (14:00 - 17:30)

■

▼

[Team Q&A with Madam] (17:30 - 20:00)

`

MEMBER 1: Project Overview, Full-Stack Architecture & Authentication

- **Role:** System Architect & Backend Infrastructure Lead
- **Time Allocated:** 3.5 Minutes (0:00 - 3:30)
- **Primary Goal:** Hook the evaluator, define why generic AI fails, present the full-stack architecture, and explain stateless user security.

1. What to Tell (Speaking Script)

> "Good morning Madam. Today, our team is presenting **SmartInterview — an AI-Powered Adaptive Mock Interview Intelligence Platform**.

>

> In today's hiring landscape, candidates face two extremes:

> 1. **Static platforms** like LeetCode or GeeksforGeeks only test syntax and static algorithms without assessing cognitive depth, technical articulation, or system tradeoffs.

> 2. **Generic AI chatbots** like ChatGPT or Claude lack pedagogical structure, hallucinate facts, and fail to track skill progression systematically.

>

> SmartInterview solves this through a novel three-pillar approach:

> - **Domain-Grounded RAG** using ChromaDB to prevent factual hallucinations.

> - A **Deterministic Bloom's Taxonomy Adaptive Engine** implemented in Python to govern cognitive difficulty progression.

> - A **Multi-Signal Evaluation Engine** combining AI critique with mathematical semantic similarity and concept coverage.

>

> Architecturally, our system uses a decoupled three-tier model:

> - **Frontend:** React 18 Single-Page Application powered by Vite and Tailwind CSS.

> - **Backend API:** Asynchronous Python FastAPI using ASGI (**`uvicorn`**) for high-throughput concurrency.

> - **Persistence:** Relational session storage in MySQL 8 via SQLAlchemy ORM, alongside a local ChromaDB vector store.

> - **Security:** Stateless authentication using JSON Web Tokens (JWT) signed with HMAC-SHA256 (HS256) and passwords salted and hashed via bcrypt."

2. How to Tell & Live Demo Flow

- **Screen Action:**

1. Open browser at **`http://localhost:5174`**.
2. Display the modern hero landing page.
3. Navigate to **Login** / **Register**, enter test credentials, and log into the **Dashboard**.

- **Delivery Tip:** Speak with authority and clarity. Highlight that our architecture is fully decoupled, scalable, and follows enterprise best practices.

3. Technical Execution Details

- FastAPI handles routing in **`backend/app/main.py`**.
- Password security: **`bcrypt.hashpw()`** generates a 60-character bcrypt hash with unique 128-bit salt.
- JWT Generation: Claims include **`sub`** (User ID), **`email`**, and **`exp`** (UTC timestamp + 24 hours).
- Protected endpoints use FastAPI's dependency injection (**`Depends(get_current_user)`**).

4. Code Walkthrough (Files to Show Madam)

- **FastAPI Main App & Middleware: `backend/app/main.py`** (Lines 14-37)
- **Authentication Endpoints: `backend/app/routers/auth.py`** (Lines 25-65)
- **Bcrypt & JWT Implementation: `backend/app/services/auth_service.py`** (Lines 20-55)
- **User ORM Entity Schema: `backend/app/models/user.py`** (Lines 10-25)

5. Expected Viva / Madam Questions & Model Answers

- **Q: Why did you pick FastAPI instead of Django or Flask?**

Answer: "FastAPI provides native asynchronous I/O via ASGI, native Pydantic data validation, and automated OpenAPI documentation. In AI applications where network requests to LLMs and vector stores take variable time, non-blocking asynchronous event loops prevent thread exhaustion."

- **Q: How are JWT tokens verified on each request?**

Answer: "Each protected request sends an **`Authorization: Bearer <token>`** header. Our **`get_current_user`** dependency decodes the token using the secret key and HS256 algorithm, validates expiration, extracts the user ID, and queries the database session."

> **Transition to Member 2:**

> *"Now, my teammate [Member 2 Name] will explain our Context Ingestion Engine and how we perform deterministic Resume and Job Description skill gap mapping."*

MEMBER 2: Context Ingestion — Resume Parsing, JD Filtering & Skill Gap Mapping

- **Role:** Data Ingestion & NLP Preprocessing Lead
- **Time Allocated:** 3.5 Minutes (3:30 - 7:00)
- **Primary Goal:** Explain deterministic document parsing, regex section cleanup, and the Group A vs Group B skill intersection logic.

1. What to Tell (Speaking Script)

> "Thank you [Member 1]. I am [Name], and I will present our **Context Ingestion and Skill Gap Analysis Pipeline**.

>

> Generic mock interview platforms ask generic questions because they lack candidate context. SmartInterview tailors every session by analyzing two essential documents: the candidate's **Resume** and the target **Job Description (JD)**.

>

> A key engineering choice we made is that **we do NOT use an LLM for document parsing**. LLM parsing is non-deterministic, slow, and expensive. Instead, our pipeline is 100% deterministic:

> 1. **High-Speed Document Extraction:** We use **PyMuPDF (`fitz`)** to extract clean text streams from PDF, DOCX, and TXT files in under 50 milliseconds.

> 2. **Regex Section Filtering:** Job descriptions contain clutter like salary, benefits, and EEO legal disclaimers. Our JD parser uses regex boundary patterns to strip non-technical sections and isolate technical requirements.

> 3. **Canonical Skill Taxonomy:** We match skills against a normalized taxonomy covering 40+ computer science disciplines.

>

> The core algorithmic innovation in my module is **Skill Grouping**:

> - **Group A ($JD \cap Resume$):** Overlapping skills found in *both* documents. The system prioritizes Group A first to verify the candidate's claimed experience against the job requirements.

> - **Group B ($JD \setminus Resume$):** Required job skills *absent* from the resume. The system tests Group B second to assess candidate adaptability and uncover skill gaps.

>

> Furthermore, we built an **Academic Syllabus Mode** allowing university students to upload course syllabi for viva and semester preparation."

2. How to Tell & Live Demo Flow

- **Screen Action:**

1. Navigate to **Resume & JD Upload** (**`/resume-upload`**).

2. Upload a sample PDF resume and paste a technical Job Description (e.g., Python Backend Engineer with SQL and Docker).

3. Show the UI extracting skills and rendering **Group A (Verified Skills)** and **Group B (Skill Gaps)** badges.

- **Delivery Tip:** Emphasize the math behind the skill sets ($A = JD \cap Resume$, $B = JD - Resume$).

3. Technical Execution Details

- Text cleaning removes non-ASCII artifacts, page numbers, and excessive whitespace.
- Section exclusion regex (**`_EXCLUDE_HEADING_PATTERNS`**) filters out headers matching **`compensation`**, **`benefits`**, **`equal opportunity`**, and **`application process`**.
- Set operations categorize skills into:

$$\text{Group A} = S_{\text{JD}} \cap S_{\text{Resume}}$$

$$\text{Group B} = S_{\text{JD}} \setminus S_{\text{Resume}}$$

4. Code Walkthrough (Files to Show Madam)

- **JD Parser & Group A/B Algorithm: `backend/app/services/jd_service.py`** (Lines 8-60, 120-155)
- **Resume Extraction Engine: `backend/app/services/resume_service.py`** (Lines 1-30) & **`scripts/parse_resume.py`**
- **Job Description API Router: `backend/app/routers/job_descriptions.py`** (Lines 40-100)
- **Syllabus Parser: `backend/app/services/syllabus_engine.py`** (Lines 30-85)

5. Expected Viva / Madam Questions & Model Answers

- **Q: What happens to skills found in the Resume but NOT in the JD?**

Answer: "Resume-only skills ($S_{\text{Resume}} \setminus S_{\text{JD}}$) are saved to the candidate's profile but excluded from the active interview plan. An interview must remain targeted to the hiring role's requirements rather than drifting into unrelated hobbies or legacy skills."

- **Q: Why use PyMuPDF instead of PyPDF2 or pdfminer?**

Answer: "PyMuPDF (**`fitz`**) is written in C (MuPDF library) and benchmarks 10x to 20x faster than pure-Python libraries like PyPDF2, while handling complex multi-column layouts and non-standard fonts reliably without crashing."

> **Transition to Member 3:**

> *"Next, my teammate [Member 3 Name] will explain our RAG knowledge base and how context-grounded interview questions are generated."*

MEMBER 3: RAG Knowledge Base & Context-Aware Question Generation

- **Role:** AI/RAG & Knowledge Engineering Lead
- **Time Allocated:** 3.5 Minutes (7:00 - 10:30)
- **Primary Goal:** Explain vector search, why RAG prevents LLM hallucinations, retrieval metrics (Hit@1, MRR), and prompt assembly.

1. What to Tell (Speaking Script)

> "Thank you [Member 2]. I am [Name], and I will discuss our **Retrieval-Augmented Generation (RAG) Architecture and Question Generation Service**.

>

> When standard LLMs generate technical interview questions, they often create questions that are overly vague, too trivial, or factually inaccurate. To guarantee technical accuracy, SmartInterview grounds all question generation in a **verified knowledge base**.

>

> Our Hybrid Graph RAG architecture is built upon four technical pillars:

> 1. **Tri-Retrieval Engine:** Instead of vanilla vector search that misses exact technical symbols or acronyms (like '2PL' or 'ACID'), we combine Dense Semantic ChromaDB, Lexical BM25Okapi keyword search, and NetworkX Knowledge Graph prerequisite traversal over 402 curated CS chunks across 8 domains.

> 2. **Reciprocal Rank Fusion (RRF, k=60):** Fuses rankings mathematically so that neither dense nor lexical retrieval dominates.

> 3. **Dense Semantic Embeddings:** The local **`all-MiniLM-L6-v2`** Sentence-BERT model generates 384-dimensional dense vectors on CPU in under 15ms.

> 4. **Empirical Retrieval Benchmarks:** Across 33 technical domain queries, our pipeline achieved 96.97% Hit@1, 100% Hit@3, and an MRR of 0.9798 with 24.6ms retrieval latency.

>

> During an interview, when a skill (like 'DBMS') is selected, we perform a cosine similarity search filtered by that domain. The top-3 reference chunks are retrieved and injected directly into a structured prompt sent to the **Groq Cloud API** running **`openai/gpt-oss-120b`**. The model is strictly instructed to generate questions grounded in those reference facts."

2. How to Tell & Live Demo Flow

- **Screen Action:**

1. Click **Start Interview** on the frontend.

2. When the question renders, point to the **Skill Badge** (e.g., **`DBMS`**), the **Difficulty Tag**, and the factually grounded technical phrasing of the question.

3. Open **`data/knowledge_stats.json`** or **`chroma_db/`** in VS Code to show the pre-indexed vector data.

- **Delivery Tip:** Highlight that our vector search runs **completely locally** without third-party vector cloud costs (like Pinecone).

3. Technical Execution Details

- Query builder: **`build_rag_query(skill, domain, bloom_level, question_type)`**.
- Semantic search executes cosine similarity over dense embeddings:

$$\text{Similarity}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|_2 \|\mathbf{d}\|_2}$$

- Thread-safe singleton pattern (`_lock = threading.Lock()`) ensures the 80MB embedding model is loaded into memory only once.

4. Code Walkthrough (Files to Show Madam)

- **RAG & Model Singleton Accessors: `backend/app/services/question_service.py`** (Lines 40-120, showing `_init_rag()`, `get_embedding_model()`, `get_chroma_collection()`)
- **Question Generation Logic: `backend/app/services/question_service.py`** (Lines 230-310)
- **Knowledge Base Statistics: `data/knowledge_stats.json`** (Lines 1-35, 402 chunks, 8 domains)
- **ChromaDB Persistent Store: `chroma_db/`** folder in workspace root

5. Expected Viva / Madam Questions & Model Answers

- **Q: Why use Groq API instead of standard OpenAI or local Ollama?**

Answer: "Groq uses custom LPUs (Language Processing Units) that achieve inference speeds exceeding 300 tokens per second. This reduces question generation and answer evaluation latency to sub-second speeds, enabling a natural, conversational interview experience."

- **Q: What is MRR (Mean Reciprocal Rank) and why is 0.9798 significant?**

Answer: "MRR evaluates where the first relevant ground-truth document appears in our retrieval results ($MRR = \frac{1}{|Q|} \sum \frac{1}{\text{rank}_i}$). An MRR of 0.9798 means the absolute best matching technical concept was returned as the #1 ranked result in almost 98% of our test queries."

> **Transition to Member 4:**

> *"Now, my teammate [Member 4 Name] will explain our Deterministic Adaptive State Machine and how Bloom's Taxonomy governs interview progression."*

MEMBER 4: Deterministic Adaptive Engine & Bloom's Taxonomy State Machine

- **Role:** Cognitive Architecture & Adaptive Algorithms Lead
- **Time Allocated:** 3.5 Minutes (10:30 - 14:00)
- **Primary Goal:** Explain why the LLM does NOT control progression, detail Bloom's 6 cognitive levels, and demonstrate the 80/50 progression state machine.

1. What to Tell (Speaking Script)

> "Thank you [Member 3]. I am [Name], and I will present the core pedagogical innovation of SmartInterview: **The Deterministic Adaptive State Machine** based on **Bloom's Taxonomy**.

>

> In many naive AI projects, developers prompt an LLM: *'Here is the user's answer, decide what question to ask next.'* This approach is deeply flawed: LLMs drift, generate erratic jumps in difficulty, and lack consistent evaluation criteria.

>

> In SmartInterview, **the LLM is never allowed to control interview progression**.

>

> Instead, progression is governed by a deterministic Python state machine modeled after **Bloom's Revised Taxonomy**, covering six progressive cognitive levels:

> 1. **Level 1 — Remember:** Factual recall and basic definitions.

> 2. **Level 2 — Understand:** Explaining principles and tradeoffs.

> 3. **Level 3 — Apply:** Implementing algorithms and practical coding scenarios.

> 4. **Level 4 — Analyze:** Debugging, architectural breakdown, and bottleneck analysis.

> 5. **Level 5 — Evaluate:** Critiquing design decisions and tradeoffs under constraints.

> 6. **Level 6 — Create:** High-level distributed system synthesis and architecture design.

>

> The state machine operates under strict mathematical thresholds based on the candidate's previous answer score:

> - **Score ≥ 80 (Mastery):** Advance to the next Bloom level (L_{i+1}). If already at Level 6, increase base difficulty (**`easy`** $\rightarrow$ **`medium`** $\rightarrow$ **`hard`**).

> - **Score $50–79$ (Adequate):** Maintain the current Bloom level to consolidate understanding.

> - **Score < 50 (Remediation):** Regress to the previous Bloom level (L_{i-1}) or provide a simpler foundational question.

>

> To ensure fairness across the entire syllabus, we use a **Weakness-Biased Round-Robin Scheduler**. It tracks running averages per skill and prioritizes struggling topics while ensuring every skill is assessed.">

> Furthermore, when a candidate completes an interview with unmastered skills, SmartInterview launches **Remedial Mastery Mode**. It performs backward depth-first search along prerequisite edges in our Knowledge Graph, identifies root-cause foundational gaps, generates dynamic AI study guides and interactive flashcards, and provides a 1-click 'Start Focused Retest' action.

2. How to Tell & Live Demo Flow

- **Screen Action:**

1. On the live interview screen (**`/interview`**), point to the current **Bloom Level Badge** (**`Remember`**).

2. Answer the question thoroughly. Once evaluated with a score ≥ 80, show the next question advancing to **`Understand`** or **`Apply`**.

3. Point out the sidebar showing real-time skill performance and question tracking.

- **Delivery Tip:** Emphasize the word **"Deterministic"** — evaluators love hearing that the business logic is predictable and robust rather than a black-box prompt.

3. Technical Execution Details

- Implemented in **`adaptive_engine.py`** via **`LearnerProfile`** and **`AdaptiveState`** dataclasses.
- Clamping function: **`clamp_bloom_order(order)`** keeps values strictly between 1 and 6.
- State transitions are serialized to JSON and persisted to the **`interview_sessions`** table after every turn, enabling session pausing and resuming.

4. Code Walkthrough (Files to Show Madam)

- **Progression Threshold Constants: `backend/app/services/adaptive_engine.py`** (Lines 38-56, showing **`ADVANCE_THRESHOLD = 80`**, **`MAINTAIN_LOWER = 50`**, **`BLOOM_QUESTION_TYPES`**)
- **State Transition Engine (`decide_next`): `backend/app/services/adaptive_engine.py`** (Lines 180-245)
- **Bloom Taxonomy Definitions: `backend/app/services/bloom.py`** (Lines 15-65)
- **Session Coordinator: `backend/app/services/interview_service.py`** (Lines 120-175)

5. Expected Viva / Madam Questions & Model Answers

- **Q: What prevents a student from getting stuck at Level 6 or Level 1 indefinitely?**

Answer: "Our clamping functions prevent out-of-bounds errors. When a candidate reaches Level 6 with a score ≥ 80, the Bloom level remains at 6 while base difficulty scales from easy to hard. If a student repeatedly scores < 50 at Level 1, our weakness-biased scheduler shifts to a different skill to avoid candidate fatigue while marking that concept for review."

- **Q: How does the system handle an interview session if the browser disconnects?**

Answer: "Because the **`AdaptiveState`** is serialized to MySQL after every question turn, the session is completely stateful. When the user reconnects, the backend reloads the exact **`LearnerProfile`**, current Bloom level, and tested skills from the database."

> **Transition to Member 5:**

> *"Now, my teammate [Member 5 Name] will conclude our presentation with our 5-Factor Answer Evaluation, Voice processing, and automated PDF reporting."*

MEMBER 5: Multi-Factor Answer Evaluation, Voice Processing & Final Analytics

- **Role:** Evaluation Systems, Audio Engineering & Analytics Lead
- **Time Allocated:** 3.5 Minutes (14:00 - 17:30)
- **Primary Goal:** Detail the 5-factor scoring formula, showcase TTS/STT voice integration, and demonstrate the dynamic PDF report generation.

1. What to Tell (Speaking Script)

> "Thank you [Member 4]. I am [Name], and I will present our **Multi-Factor Answer Evaluation Service, Voice Pipeline, and Student Performance Analytics**.

>

> In many mock interview platforms, scoring is arbitrary: an LLM is asked for a single number between 1 and 100, which introduces high variance.

>

> SmartInterview uses a **Multi-Factor Scoring Engine** computed via a strictly defined weighted mathematical formula:

>

> $$\text{Overall Score} = 0.30(T) + 0.20(C) + 0.20(R) + 0.15(S) + 0.15(K)$$

>

> *Where:*

> - **T (Technical Correctness — 30%):** Assessed via Groq LLM at a low temperature of 0.3 for consistency.

> - **C (Completeness — 20%):** Measures whether all facets of the prompt were addressed.

> - **R (Relevance — 20%):** Evaluates focus and conciseness.

> - **S (Semantic Similarity — 15%):** Measured via local SentenceTransformer cosine similarity against ground-truth technical definitions.

> - **K (Concept Coverage — 15%):** The ratio of expected technical keywords and concepts identified in the response.

>

> Next, for our **Voice Architecture**:

> - **Text-to-Speech (TTS):** We use Microsoft Neural TTS via **`edge-tts`** (**`en-US-ChristopherNeural`**). It strips markdown formatting, normalizes technical terms, and uses an in-memory SHA-256 LRU cache for near-instant audio replay.

> - **Speech-to-Text (STT):** Powered by Groq Whisper (**`whisper-large-v3`**) primed with a domain-specific technical vocabulary prompt to accurately transcribe technical terms like 'Kubernetes', 'polymorphism', or 'ACID'.

>

> Finally, when the session ends, our **ReportLab engine** compiles an official multi-page downloadable PDF report featuring Bloom cognitive progression charts, skill strengths and weaknesses, and personalized remediation advice."

2. How to Tell & Live Demo Flow

• **Screen Action:**

1. Click the **Speaker icon** next to the interview question to demonstrate the crisp, natural TTS voice.

2. Type or speak an answer, click **Submit Answer**, and show the multi-dimensional breakdown scores appearing immediately.

3. Click **Finish Interview** to transition to the **Results Dashboard** (`/results`).

4. Click **Download PDF Report** to download the professional PDF report and open it to show Madam.

• **Delivery Tip:** Open the PDF report on screen — evaluators love tangible outputs like downloadable student reports.

3. Technical Execution Details

• Evaluator prompt enforces structured JSON output parsed into **`EvaluationResult`** dataclass.

• Audio streaming: In-memory byte streams mean candidate voice recordings are never written to disk, preserving privacy.

• PDF generation uses a custom **`NumberedCanvas`** in ReportLab to perform two-pass rendering for running headers, footers, and page numbers.

4. Code Walkthrough (Files to Show Madam)

• **Scoring Weights & Evaluation Engine: `backend/app/services/evaluation_service.py`** (Lines 36-65, showing **`WEIGHT_TECHNICAL = 0.30`**, **`WEIGHT_COMPLETENESS = 0.20`**, **`WEIGHT_SEMANTIC_SIMILARITY = 0.15`**)

• **Voice Engine (TTS & Whisper STT): `backend/app/services/voice_service.py`** (Lines 27-65, showing **`TECHNICAL_VOCABULARY_PROMPT`** and **`clean_text_for_speech`**)

• **PDF Report Generator: `backend/app/services/report_service.py`** (Lines 54-115, showing **`NumberedCanvas`** and ReportLab tables)

• **Frontend Results Dashboard: `frontend/src/pages/Results.jsx`** (Lines 40-110)

5. Expected Viva / Madam Questions & Model Answers

• **Q: Why combine Semantic Similarity with LLM evaluation?**

Answer: "An LLM can occasionally be swayed by verbose phrasing. Mathematical cosine similarity using SentenceTransformers compares candidate embeddings against verified reference knowledge, providing an objective, deterministic check that balances the LLM's qualitative feedback."

• **Q: How does your PDF generator handle tables or text that span multiple pages?**

Answer: "We use ReportLab Platypus flowable elements (**`Paragraph`**, **`Table`**, **`Spacer`**, **`KeepTogether`**) with automatic pagination, coupled with a two-pass **`NumberedCanvas`** that dynamically computes total pages."

3.1 355-Test Verification Matrix Across All 18 Phases

SmartInterview ensures rock-solid architectural stability through **355 unit and integration tests across 18 phases (100% pass rate in 87.90s)**:

• Phase 1 (Dense RAG): 25 tests

• Phase 2–4 (Knowledge Graph): 69 tests

• Phase 5 (BM25 Engine): 34 tests

• Phase 6–7 (Graph Traversal & RRF): 48 tests

• Phase 8 (Reranker & Scoring): 36 tests

- Phase 9–10 (Pipeline & Bloom Progression): 39 tests
- Phase 11–13 (Security & Golden Dataset): 48 tests
- Phase 14–18 (Baseline & Safeguards): 56 tests
- **Total: 355 Passing Tests**

> **Conclusion (Member 5 to Panel):**

> *"To conclude, SmartInterview combines deterministic NLP parsing, verified RAG vector grounding, a pedagogical Bloom's state machine, and multi-factor evaluation to deliver a production-ready mock interview platform. We would be delighted to answer any questions. Thank you!"*

Part 4: Emergency Live Demo Cheat Sheet

Follow these 6 steps sequentially during your demonstration:

1. **Step 1 (Member 1):** Open **`http://localhost:5174`**. Show Landing Page, click Login, and log in to the Dashboard.

2. **Step 2 (Member 2):** Navigate to Resume & JD Upload. Upload a sample resume PDF, paste a job description, and show **Group A** and **Group B** badges.

3. **Step 3 (Member 3):** Click Start Interview. Point out the dynamically loaded technical question with the domain skill tag.

4. **Step 4 (Member 4):** Click the speaker icon to hear the question. Submit a strong technical answer; show the score ≥ 80 and observe the next question advancing in Bloom cognitive level.

5. **Step 5 (Member 5):** Submit a second answer to show continuous multi-factor evaluation feedback.

6. **Step 6 (Member 5):** Click End Session, view the Results analytics page, and click **Download PDF Report** to show the final document.

Part 5: Master File & Directory Lookup Table

When your madam asks: *"Open the file where this is implemented"*, use this quick guide:

| Feature / Module | Exact File Path | Key Functions / Classes |

|---|---|---|

| **FastAPI App & CORS** | **`backend/app/main.py`** | **`app = FastAPI()`**, **`CORSMiddleware`** |

| **Auth & Security** | **`backend/app/services/auth_service.py`** | **`verify_password()`**, **`create_access_token()`** |

| **User Entity Model** | **`backend/app/models/user.py`** | **`class User(Base)`** |

| **JD & Skill Mapping** | **`backend/app/services/jd_service.py`** | **`_EXCLUDE_HEADING_PATTERNS`**, Group A/B logic |

| **Resume Extraction** | **`backend/app/services/resume_service.py`** | PyMuPDF text extraction |

| **RAG & Singletons** | **`backend/app/services/question_service.py`** | **`_init_rag()`**, **`get_embedding_model()`**, **`call_groq()`** |

| **Vector Stats** | **`data/knowledge_stats.json`** | 402 chunks, 8 domains metadata |

| **Bloom Taxonomy** | **`backend/app/services/bloom.py`** | **`REMEMBER`** to **`CREATE`**, **`clamp_bloom_order()`** |

| **Adaptive State Machine** | **`backend/app/services/adaptive_engine.py`** | **`ADVANCE_THRESHOLD = 80`**, **`decide_next()`** |

| **Answer Scoring** | **`backend/app/services/evaluation_service.py`** | **`WEIGHT_TECHNICAL`**, **`evaluate_answer()`** |

| **Voice TTS & STT** | **`backend/app/services/voice_service.py`** | **`synthesize_speech_edge()`**, **`transcribe_audio()`** |

| **PDF Report Service** | **`backend/app/services/report_service.py`** | **`generate_interview_report()`**, **`NumberedCanvas`** |

| **React Interview View** | **`frontend/src/pages/Interview.jsx`** | Audio playback, question rendering, answer submit |

| **React Results View** | **`frontend/src/pages/Results.jsx`** | Performance breakdown, PDF trigger |